Multiplicación de Matrices en CUDA

https://gemini.google.com/app/6c76f89a5c39b16c

User prompt: Dame un programa en c++ que haga operaciones matricules y se se ejecute en cuda envidia

Response: Este programa implementa la **multiplicación de dos matrices cuadradas** ($C = A \times B$) utilizando **C++ y CUDA** con memoria compartida (*shared memory*) por bloques (*tiling*), lo que optimiza significativamente el acceso a la memoria global de la GPU.

Código Completo (`matrix_mul.cu`)

```
#include <iostream>
#include <vector>
#include <cmath>
#include <cuda_runtime.h>

// Tamaño del bloque de hilos (TILE_SIZE x TILE_SIZE)
#define TILE_SIZE 16

// Kernel de CUDA para multiplicación de matrices usando Tiling (Memoria Compartida)
__global__ void matrixMulKernel(const float* A, const float* B, float* C, int N) {
    // Memoria compartida para los bloques de A y B
    __shared__ float ds_A[TILE_SIZE][TILE_SIZE];
    __shared__ float ds_B[TILE_SIZE][TILE_SIZE];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    // Fila y columna de la matriz C asignadas a este hilo
    int row = by * TILE_SIZE + ty;
    int col = bx * TILE_SIZE + tx;

    float pValue = 0.0f;

    // Iterar sobre las submatrices (tiles) necesarias para calcular C[row][col]
    for (int ph = 0; ph < (N + TILE_SIZE - 1) / TILE_SIZE; ++ph) {
        // Cargar datos a la memoria compartida con verificación de límites
        if (row < N && (ph * TILE_SIZE + tx) < N)
            ds_A[ty][tx] = A[row * N + ph * TILE_SIZE + tx];
        else
            ds_A[ty][tx] = 0.0f;

        if (col < N && (ph * TILE_SIZE + ty) < N)
            ds_B[ty][tx] = B[(ph * TILE_SIZE + ty) * N + col];
        else
            ds_B[ty][tx] = 0.0f;

        // Sincronizar hilos para asegurar que toda la memoria compartida esté cargada
        __syncthreads();

        // Multiplicar las dos submatrices cargadas
        for (int k = 0; k < TILE_SIZE; ++k) {
            pValue += ds_A[ty][k] * ds_B[k][tx];
        }

        // Sincronizar antes de cargar la siguiente submatriz
        __syncthreads();
    }

    // Escribir el resultado en la matriz C
    if (row < N && col < N) {
        C[row * N + col] = pValue;
    }
}

// Función auxiliar para verificar errores de CUDA
void checkCudaError(cudaError_t err, const char* msg) {
    if (err != cudaSuccess) {
        std::cerr << "Error CUDA (" << msg << "): " << cudaGetErrorString(err) << std::endl;
        exit(EXIT_FAILURE);
    }
}

int main() {
```

```
    int N = 1024; // Matriz de 1024 x 1024
    size_t bytes = N * N * sizeof(float);

    // 1. Asignar memoria en el Host (CPU)
    std::vector<float> h_A(N * N, 1.5f);
    std::vector<float> h_B(N * N, 2.0f);
    std::vector<float> h_C(N * N, 0.0f);

    // 2. Asignar memoria en el Device (GPU)
    float *d_A, *d_B, *d_C;
    checkCudaError(cudaMalloc(&d_A, bytes), "cudaMalloc A");
    checkCudaError(cudaMalloc(&d_B, bytes), "cudaMalloc B");
    checkCudaError(cudaMalloc(&d_C, bytes), "cudaMalloc C");

    // 3. Copiar datos del Host a la GPU
    checkCudaError(cudaMemcpy(d_A, h_A.data(), bytes, cudaMemcpyHostToDevice), "Memcpy A");
    checkCudaError(cudaMemcpy(d_B, h_B.data(), bytes, cudaMemcpyHostToDevice), "Memcpy B");

    // 4. Configurar la rejilla de bloques e hilos
    dim3 dimBlock(TILE_SIZE, TILE_SIZE);
    dim3 dimGrid((N + TILE_SIZE - 1) / TILE_SIZE, (N + TILE_SIZE - 1) / TILE_SIZE);

    std::cout << "Ejecutando kernel de multiplicación (" << N << "x" << N << ")..." << std::endl;

    // 5. Lanzar el Kernel
    matrixMulKernel<<<dimGrid, dimBlock>>>(d_A, d_B, d_C, N);
    checkCudaError(cudaGetLastError(), "Lanzamiento de Kernel");
    checkCudaError(cudaDeviceSynchronize(), "cudaDeviceSynchronize");

    // 6. Copiar resultados de la GPU al Host
    checkCudaError(cudaMemcpy(h_C.data(), d_C, bytes, cudaMemcpyDeviceToHost), "Memcpy C");

    // 7. Verificación rápida de resultado (C[0] debe ser N * 1.5 * 2.0 = 3072)
    std::cout << "Resultado C[0]: " << h_C[0] << " (Esperado: " << N * 1.5f * 2.0f << ")" << std::endl;

    // 8. Liberar memoria GPU
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}
```

Compilación y Ejecución

Para compilar con el compilador de NVIDIA (`nvcc`):

```
nvcc -O3 matrix_mul.cu -o matrix_mul
./matrix_mul
```

Puntos clave del diseño:

- **Tiling (Memoria Compartida):** Reduce la carga sobre el ancho de banda de la memoria global cargando bloques de tamaño `16x16` (`TILE_SIZE`) a la memoria SRAM de la GPU (`__shared__`).
- **Manejo de bordes:** Incluye guardas `if (row < N && col < N)` para que funcione correctamente incluso si la dimensión N no es múltiplo exacto de `TILE_SIZE`.
- **Manejo de errores:** La función `checkCudaError` facilita la depuración de llamadas a la API de CUDA.

User prompt: ¿Cómo modifico el programa CUDA de multiplicación de matrices para soportar dimensiones no cuadradas M x N x K y medir el rendimiento en GFLOPS?

Response: Para soportar dimensiones no cuadradas, calculamos el producto $C_{M \times N} = A_{M \times K} \times B_{K \times N}$.

Además, para medir el rendimiento real en **GFLOPS** (Giga Floating-point Operations Per Second), utilizamos la API de eventos nativa de CUDA (`cudaEvent_t`), la cual mide únicamente el tiempo transcurrido en la GPU excluyendo latencias de la CPU.

Código Actualizado (`matrix_mul_arbitrary.cu`)

```
#include <iostream>
#include <vector>
```

```
#include <cmath>
#include <cuda_runtime.h>

#define TILE_SIZE 16

// Kernel para multiplicación de matrices no cuadradas: A (M x K) * B (K x N) = C (M x N)
__global__ void matrixMulKernel(const float* A, const float* B, float* C, int M, int K, int N) {
    __shared__ float ds_A[TILE_SIZE][TILE_SIZE];
    __shared__ float ds_B[TILE_SIZE][TILE_SIZE];

    int bx = blockIdx.x;  int by = blockIdx.y;
    int tx = threadIdx.x; int ty = threadIdx.y;

    // Fila en A/C (rango 0 a M-1) y Columna en B/C (rango 0 a N-1)
    int row = by * TILE_SIZE + ty;
    int col = bx * TILE_SIZE + tx;

    float pValue = 0.0f;

    // Recorrer la dimensión compartida K en bloques de TILE_SIZE
    for (int ph = 0; ph < (K + TILE_SIZE - 1) / TILE_SIZE; ++ph) {
        // Carga con guardas de seguridad para dimensiones arbitrarias
        if (row < M && (ph * TILE_SIZE + tx) < K)
            ds_A[ty][tx] = A[row * K + ph * TILE_SIZE + tx];
        else
            ds_A[ty][tx] = 0.0f;

        if (col < N && (ph * TILE_SIZE + ty) < K)
            ds_B[ty][tx] = B[(ph * TILE_SIZE + ty) * N + col];
        else
            ds_B[ty][tx] = 0.0f;

        __syncthreads();

        for (int k = 0; k < TILE_SIZE; ++k) {
            pValue += ds_A[ty][k] * ds_B[k][tx];
        }

        __syncthreads();
    }

    // Guardar resultado final si está dentro de los límites M x N
    if (row < M && col < N) {
        C[row * N + col] = pValue;
    }
}

void checkCudaError(cudaError_t err, const char* msg) {
    if (err != cudaSuccess) {
        std::cerr << "Error CUDA (" << msg << "): " << cudaGetErrorString(err) << std::endl;
        exit(EXIT_FAILURE);
    }
}

int main() {
    // Dimensiones arbitrarias (M x K) * (K x N) = (M x N)
    int M = 2048;
    int K = 1024;
    int N = 4096;

    size_t bytes_A = M * K * sizeof(float);
    size_t bytes_B = K * N * sizeof(float);
    size_t bytes_C = M * N * sizeof(float);

    // 1. Memoria en Host
    std::vector<float> h_A(M * K, 1.5f);
    std::vector<float> h_B(K * N, 2.0f);
    std::vector<float> h_C(M * N, 0.0f);

    // 2. Memoria en Device
    float *d_A, *d_B, *d_C;
    checkCudaError(cudaMalloc(&d_A, bytes_A), "cudaMalloc A");
    checkCudaError(cudaMalloc(&d_B, bytes_B), "cudaMalloc B");
    checkCudaError(cudaMalloc(&d_C, bytes_C), "cudaMalloc C");
```

```
    checkCudaError(cudaMemcpy(d_A, h_A.data(), bytes_A, cudaMemcpyHostToDevice), "Memcpy A");
    checkCudaError(cudaMemcpy(d_B, h_B.data(), bytes_B, cudaMemcpyHostToDevice), "Memcpy B");

    // 3. Mapeo de hilos y bloques en 2D (Eje X -> Columnas N, Eje Y -> Filas M)
    dim3 dimBlock(TILE_SIZE, TILE_SIZE);
    dim3 dimGrid((N + TILE_SIZE - 1) / TILE_SIZE, (M + TILE_SIZE - 1) / TILE_SIZE);

    // 4. Configurar eventos de CUDA para medición precisa del tiempo
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    std::cout << "Ejecutando multiplicación: (" << M << "x" << K << ") * (" << K << "x" << N << ")..." << std:

    // Registrar inicio
    cudaEventRecord(start);

    // Lanzar kernel
    matrixMulKernel<<<dimGrid, dimBlock>>>(d_A, d_B, d_C, M, K, N);

    // Registrar fin y sincronizar
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    // Calcular tiempo transcurrido en milisegundos
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    // 5. Copiar resultado
    checkCudaError(cudaMemcpy(h_C.data(), d_C, bytes_C, cudaMemcpyDeviceToHost), "Memcpy C");

    // 6. Cálculo del Rendimiento en GFLOPS
    // Cada elemento de C requiere K multiplicaciones y K sumas -> 2 * M * N * K operaciones flotantes
    double flops = 2.0 * static_cast<double>(M) * static_cast<double>(N) * static_cast<double>(K);
    double seconds = milliseconds / 1000.0;
    double gflops = (flops / 1e9) / seconds;

    std::cout << "\n--- Métricas de Rendimiento ---" << std::endl;
    std::cout << "Tiempo de Kernel : " << milliseconds << " ms" << std::endl;
    std::cout << "Rendimiento      : " << gflops << " GFLOPS" << std::endl;
    std::cout << "Verificación C[0]: " << h_C[0] << " (Esperado: " << K * 1.5f * 2.0f << ")" << std::endl;

    // Limpieza
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}
```

Cambios Clave Explicados

1. **Gestión de Índices en $M \times K \times N$:**
   - La rejilla (`dimGrid`) mapea las columnas N al eje X y las filas M al eje Y.
   - La condición de carga a la memoria compartida evalúa los límites de M para A, de N para B, y la dimensión K para los bucles intermedios.
2. **Cálculo exacto de Operaciones FLOP:**
   - Para calcular cada celda de la matriz resultado C, se efectúan K multiplicaciones y K adiciones ($2 \times K$ FLOPs).
   - Para la matriz completa de $M \times N$, el número total de operaciones es $2 \cdot M \cdot N \cdot K$.
3. **Uso de Eventos CUDA (`cudaEvent_t`):**
   - Medir tiempos en la CPU con `std::chrono` suele introducir sobrecoste por llamadas asíncronas. `cudaEventRecord` mide la duración exacta execution-to-execution en los SMs (*Streaming Multiprocessors*) de la GPU.
